DataFrame_CSV

May 25, 2026

1 Working with DataFrame and CSV

CSV stands for **Comma separated Values**. These files are easy to manipulate, and can be used across multiple applications. Example: DataFrame, SQL, etc.

The default delimiter in CSV is `comma ','`. But, we can use different delimiters in our CSV files. We have to mention the delimiter, except for `comma`.

[ ]:
```
# Import a CSV file into DataFrame
import pandas as pd

df = pd.read_csv("data.csv")
print(df)
```

```
   RollNo  Name  Age
0     101  Ramu   16
1     102  Sonu   19
2     103  Rita   16
3     104  Amit   20
```

By default, the first row of the CSV file is always treated as the `header row`.

[ ]:
```
# If we consider the second row as our header row
df = pd.read_csv('data.csv', header=1)
print(df)
```

```
   101  Ramu  16
0  102  Sonu  19
1  103  Rita  16
2  104  Amit  20
```

[ ]:
```
df = pd.read_csv('data.csv', header=2)
print(df)
```

```
   102  Sonu  19
0  103  Rita  16
1  104  Amit  20
```

[ ]:
```
df = pd.read_csv("data.csv")
print(df)
```

```
   RollNo  Name  Age
0     101  Ramu   16
1     102  Sonu   19
2     103  Rita   16
3     104  Amit   20
```

[ ]:
```
df = pd.read_csv("data.csv", index_col='RollNo')
print(df)
```

```
        Name  Age
RollNo
101     Ramu   16
102     Sonu   19
103     Rita   16
104     Amit   20
```

Instead of putting a new system-defined index in our DataFrame, we are considering `RollNo` as our index values. This same code can be written as follows:

[ ]:
```
df = pd.read_csv("data.csv", index_col=0)
print(df)
```

```
        Name  Age
RollNo
101     Ramu   16
102     Sonu   19
103     Rita   16
104     Amit   20
```

If we make `Age` as our index value, then:

[ ]:
```
df = pd.read_csv("data.csv", index_col=2)
print(df)
```

```
     RollNo  Name
Age
16      101  Ramu
19      102  Sonu
16      103  Rita
20      104  Amit
```

[ ]:
```
df = pd.read_csv("data.csv")
print(df)
```

```
   RollNo  Name  Age
0     101  Ramu   16
1     102  Sonu   19
2     103  Rita   16
3     104  Amit   20
```

1.1 Filter the CSV file value:

Finding details of those students whose `Age` is more than 16.

[ ]:
```
age_filter = df[df['Age']>16]
print(age_filter)
```

```
   RollNo  Name  Age
1     102  Sonu   19
3     104  Amit   20
```

[ ]:
```
print(type(age_filter))
```

```
<class 'pandas.core.frame.DataFrame'>
```

1.2 Store the filtered DataFrame to a CSV file

[ ]:
```
age_filter.to_csv('filter_df.csv')
```

If we want to ignore the index values of the DataFrame, then:

[ ]:
```
age_filter.to_csv('filter_df.csv',index=False)
```

Setting `index=False` ignores the index which is present in the DataFrame.